

Prueba de Caja Blanca

“REGISTRO DE VIAJES TRANS LARA IVAN EXPRESS”

Integrantes:

Kerlly Bonilla

Darwin Cuchiye

Fecha: 2025/07/22

Prueba caja blanca de Requisito N° 1: Creación de Registros de Viaje: El programa debe permitir a los usuarios agregar nuevos registros de viaje. Esto incluye la captura de datos como ID, conductor, empresa, origen, destino, fecha de salida, fecha de retorno y costo; a su vez realizar una validación para corroborar que la información solicitada sea la pertinente.

1. CÓDIGO FUENTE

```
void crearRegistro(Registro *r, Registro lista[], int cantidad) { printf("\n==== FORMULARIO DE
NUEVO REGISTRO DE VIAJE =====\n"); char temp_input[100]; // Buffer temporal para inputs

// 1. ID del viaje - VALIDACIONES: Longitud, No vacío, Único
while (1) {

    printf("1. Ingrese el ID unico del viaje (ej. VJE001, maximo 6 caracteres):
");
    fgets(temp_input, sizeof(temp_input), stdin);
    temp_input[strcspn(temp_input, "\n")] = '\0';
    strcpy(r->id, temp_input); // Copiar al registro para validar

    if (strlen(r->id) == 0) {
        printf("Error: El ID no puede estar vacio.\n");
    } else if (strlen(r->id) > 6) { // Si el array es de 8, 7 chars + null. Si
pedimos max 6, es 6 + null
        printf("Error: El ID excede el maximo de 6 caracteres.\n");
    } else if (buscarPorID(lista, cantidad, r->id) != -1) {
        printf("Error: El ID '%s' ya existe. Por favor, ingrese un ID
unico.\n", r->id);
    } else {
        break; // ID válido
    }
}

// 2. Conductor - VALIDACION: No vacío
while (1) {
    printf("2. Ingrese el nombre completo del conductor asignado (maximo 49
caracteres): ");
    fgets(r->conductor, sizeof(r->conductor), stdin);
    r->conductor[strcspn(r->conductor, "\n")] = '\0';
    if (strlen(r->conductor) == 0) {
        printf("Error: El nombre del conductor no puede estar vacio.\n");
    } else {
        break;
    }
}

// 3. Empresa - VALIDACION: No vacío
while (1) {
    printf("3. Ingrese el nombre de la empresa a la que se realizo el viaje
(maximo 49 caracteres): ");
    fgets(r->empresa, sizeof(r->empresa), stdin);
    r->empresa[strcspn(r->empresa, "\n")] = '\0';
    if (strlen(r->empresa) == 0) {
        printf("Error: El nombre de la empresa no puede estar vacio.\n");
    } else {
        break;
    }
}

// 4. Origen - VALIDACION: No vacío
while (1) {
    printf("4. Ingrese la ciudad de origen del viaje (ej. Quito, maximo 49
caracteres): ");
```

```

fgets(r->origen, sizeof(r->origen), stdin);
r->origen[strcspn(r->origen, "\n")] = '\0';
if (strlen(r->origen) == 0) {
    printf("Error: El origen no puede estar vacio.\n");
} else {
    break;
}
}

// 5. Destino - VALIDACION: No vacío
while (1) {
    printf("5. Ingrese la ciudad de destino del viaje (ej. Guayaquil, maximo 49
caracteres): ");
    fgets(r->destino, sizeof(r->destino), stdin);
    r->destino[strcspn(r->destino, "\n")] = '\0';
    if (strlen(r->destino) == 0) {
        printf("Error: El destino no puede estar vacio.\n");
    } else {
        break;
    }
}

// 6. Fecha de salida - VALIDACIONES: Formato YYYY-MM-DD, No vacío
while (1) {
    printf("6. Ingrese la fecha de salida del viaje (Formato YYYY-MM-DD, maximo
49 caracteres): ");
    fgets(r->fecha_salida, sizeof(r->fecha_salida), stdin);
    r->fecha_salida[strcspn(r->fecha_salida, "\n")] = '\0';
    if (strlen(r->fecha_salida) == 0) {
        printf("Error: La fecha de salida no puede estar vacia.\n");
    } else if (!validarFecha(r->fecha_salida)) {
        printf("Error: Formato de fecha invalido. Use YYYY-MM-DD.\n");
    } else {
        break;
    }
}

// 7. Fecha de retorno - VALIDACIONES: Formato YYYY-MM-DD, No vacío, Posterior
a fecha de salida
while (1) {
    printf("7. Ingrese la fecha de retorno del viaje (Formato YYYY-MM-DD,
maximo 49 caracteres): ");
    fgets(r->fecha_retorno, sizeof(r->fecha_retorno), stdin);
    r->fecha_retorno[strcspn(r->fecha_retorno, "\n")] = '\0';
    if (strlen(r->fecha_retorno) == 0) {
        printf("Error: La fecha de retorno no puede estar vacia.\n");
    } else if (!validarFecha(r->fecha_retorno)) {
        printf("Error: Formato de fecha invalido. Use YYYY-MM-DD.\n");
    }
    if (strcmp(r->fecha_retorno, r->fecha_salida) < 0) {
        printf("Error: La fecha de retorno no puede ser anterior a la fecha de
salida.\n");
    }
    else{
        break;
    }
}

// 8. Costo - VALIDACIONES: Solo números enteros, no negativos
printf("8. Ingrese el costo total del viaje (solo numeros enteros no
negativos): ");
while (scanf("%d", &r->costo) != 1 || r->costo < 0) {
    printf("Entrada invalida. Por favor, ingrese solo numeros enteros no
negativos para el costo: ");
}

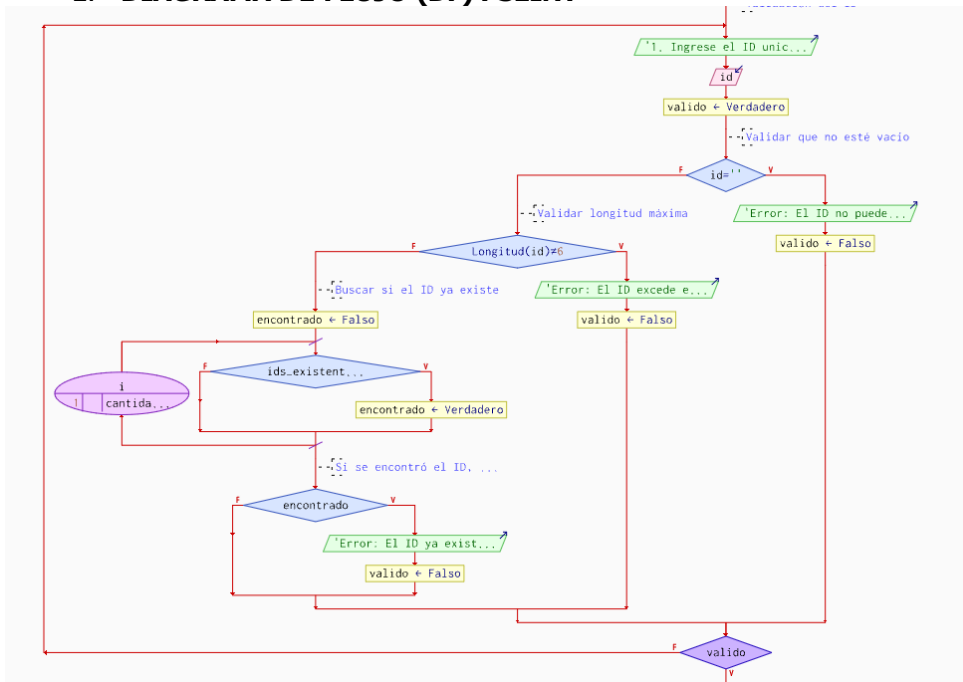
```

```

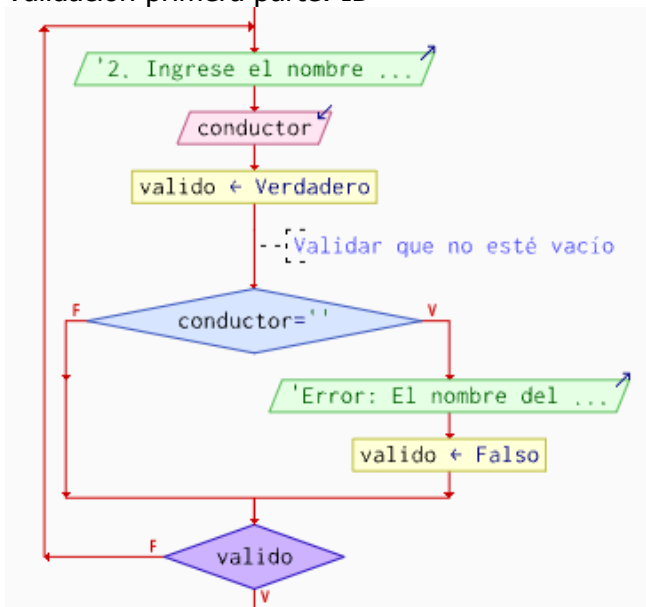
limpiarBuffer(); // Limpia el bufer si el usuario ingresa algo que no es un
numero
}
limpiarBuffer(); // Limpia el buffer después de scanf
printf("\nRegistro de viaje creado con exito!\n");
}

```

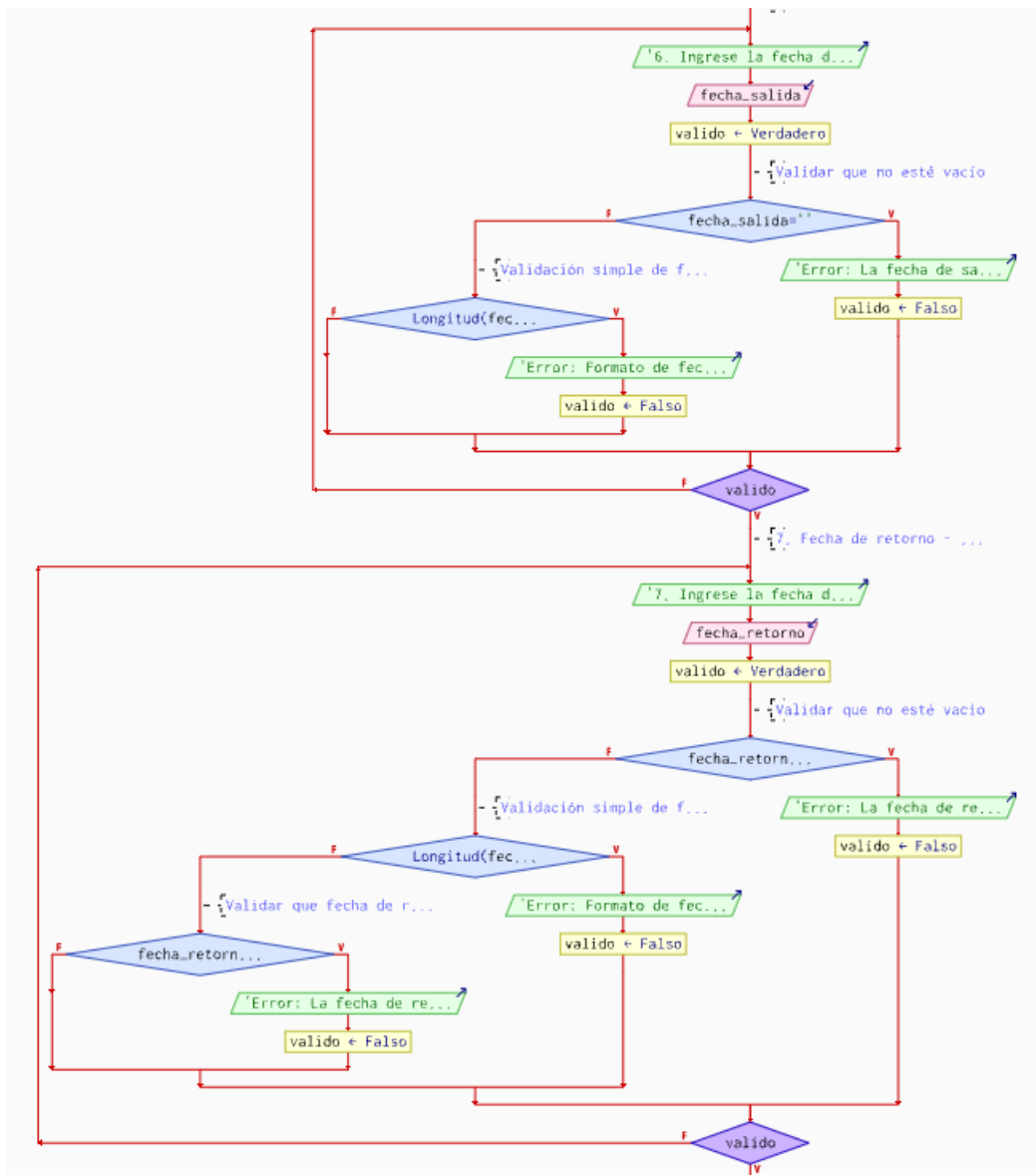
2. DIAGRAMA DE FLUJO (DF) PSEINT



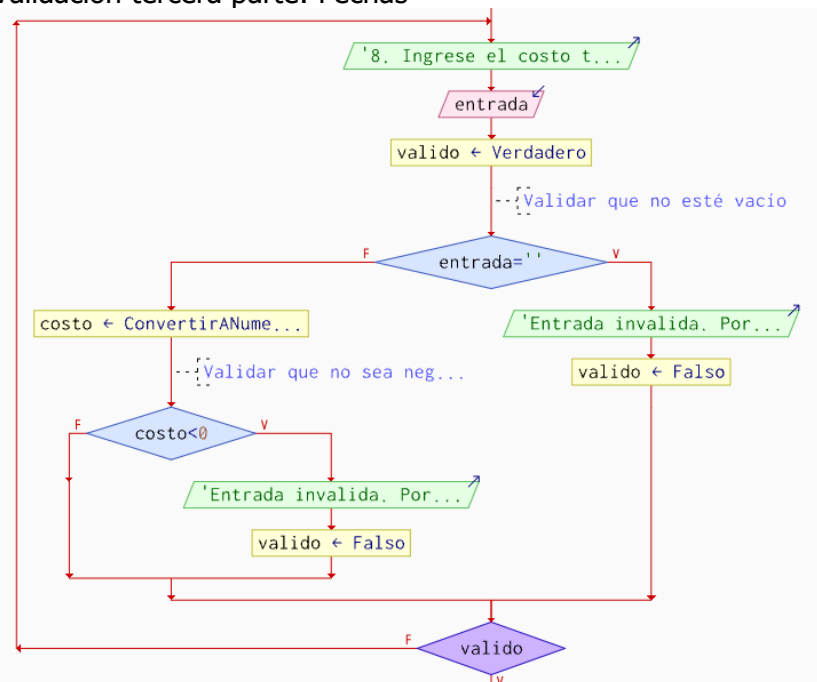
Validación primera parte: ID



Validación segunda parte: Conductor, Empresa, Origen y Destino



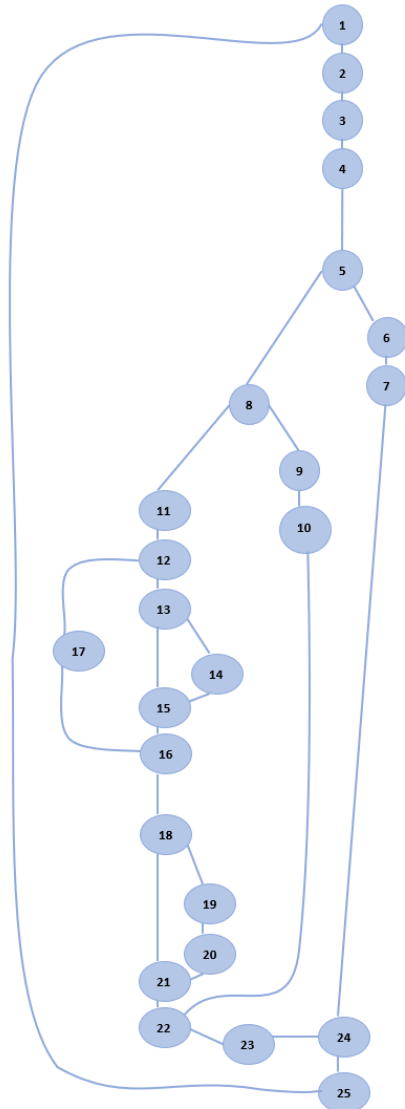
Validación tercera parte: Fechas



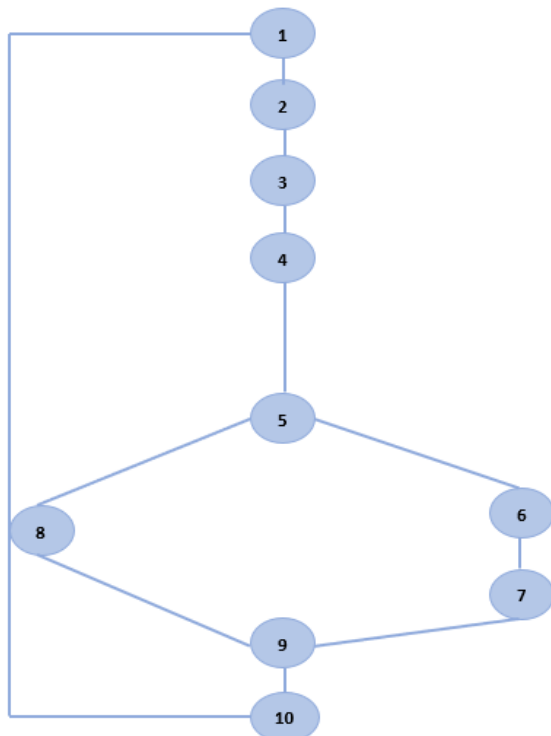
Validación parte 4: Costo

3. GRAFO DE FLUJO (GF)

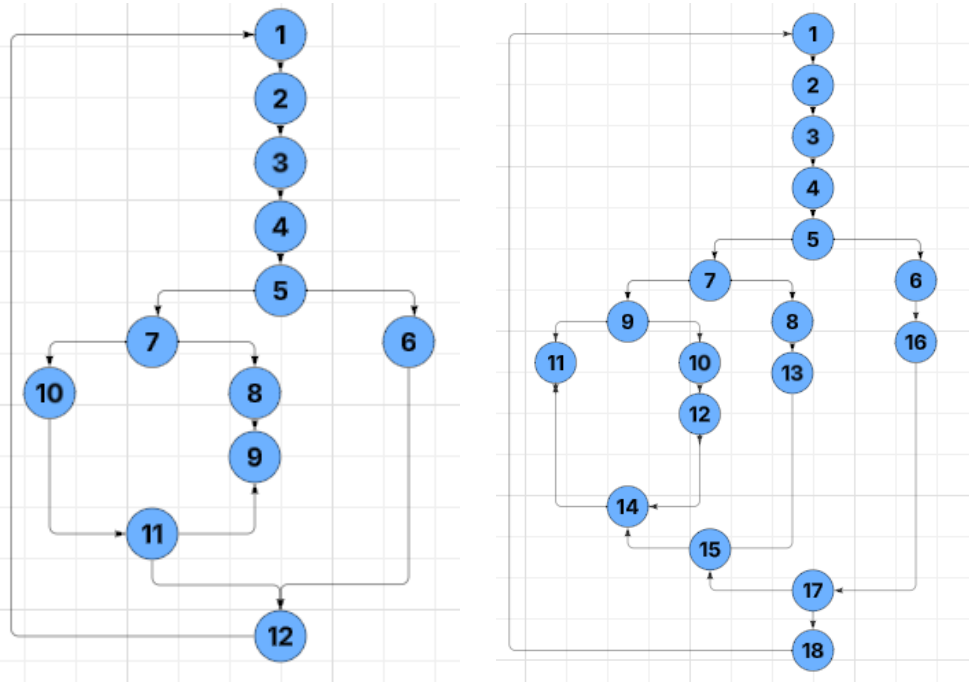
Primera Parte



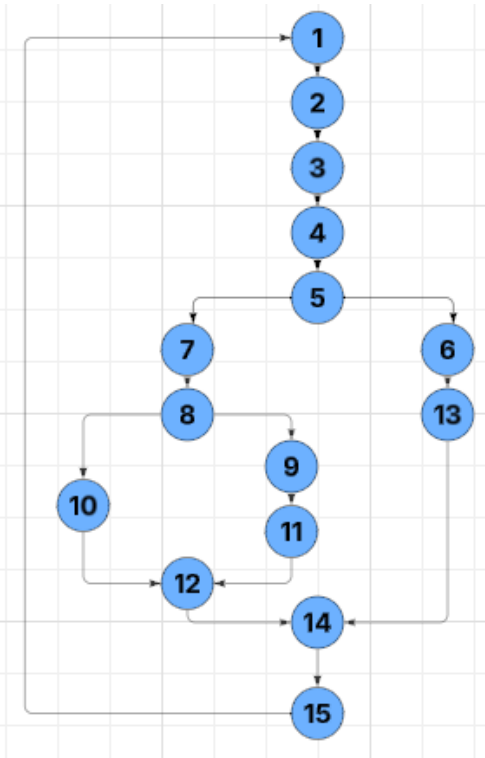
Segunda Parte



Tercera Parte



Cuarta Parte



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico)

Prueba 1 RUTAS:

R1: 1→2→3→4→5→6→7→24→25

R2: 1→2→3→4→5→8→9→10→22→23→24→25

R3: 1→2→3→4→5→8→11→12→13→14→15→16→18→19→20→21→22→23→24→25

R4: 1→2→3→4→5→8→11→12→13→15→16→18→19→20→21→22→23→24→25

R5: 1→2→3→4→5→8→11→12→13→15→16→18→21→22→23→24→25

Prueba 2 RUTAS:

R1: 1→2→3→4→5→6→7→9→10

R2: 1→2→3→4→5→8→9→10

Prueba 3.1 RUTAS:

R1: 1→2→3→4→5→6→12

R2: 1→2→3→4→5→7→8→9→11→12

R3: 1→2→3→4→5→7→10→11→12

Prueba 3.2 RUTAS:

R1: 1→2→3→4→5→6→16→17→18

R2: 1→2→3→4→5→7→8→13→15→17→18

R3: 1→2→3→4→5→7→9→10→12→14→15→17→18

R4: 1→2→3→4→5→7→9→11→14→15→17→18

Prueba 4 RUTAS:

R1: 1→2→3→4→5→6→13→14→15

R2: 1→2→3→4→5→7→8→9→11→12→14→15

R3: 1→2→3→4→5→7→8→10→12→14→15

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

Prueba 1.-

- $V(G) = \text{número de nodos predichados(decisiones)} + 1$
 $V(G) = 4 + 1 = 5$
- $V(G) = A - N + 2$
 $V(G) = 28 - 25 + 2 = 5$

Prueba2.-

- $V(G) = \text{número de nodos predichados(decisiones)} + 1$ $V(G) = 1 + 1 = 2$
- $V(G) = A - N + 2$
 $V(G) = 10 - 12 + 2 = 2$

Prueba 3.1.-

- $V(G) = \text{número de nodos predichados(decisiones)} + 1$ $V(G) = 2 + 1 = 3$
- $V(G) = A - N + 2$
 $V(G) = 13 - 12 + 2 = 3$
-

Prueba 3.2.-

- $V(G) = \text{número de nodos predichados(decisiones)} + 1$ $V(G) = 3 + 1 = 4$
- $V(G) = A - N + 2$
 $V(G) = 20 - 18 + 2 = 4$

Prueba 4.-

- $V(G) = \text{número de nodos predichados(decisiones)} + 1$ $V(G) = 2 + 1 = 3$
- $V(G) = A - N + 2$

$$V(G)=16-15+2=3$$

DONDE:

P: Número de nodos predicado

A: Número de aristas

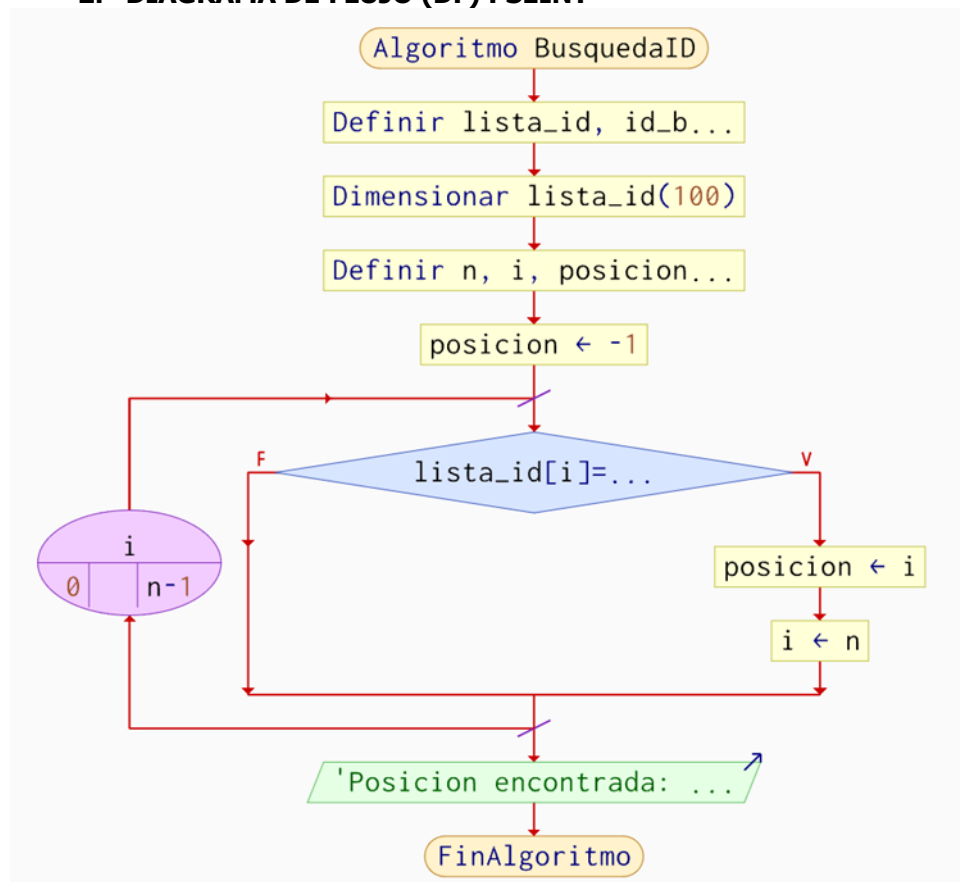
N: Número de nodos

Prueba caja blanca de Requisito N° 2: El programa debe permitir la búsqueda de registros mediante su ID único, desplegando toda la información pertinente.

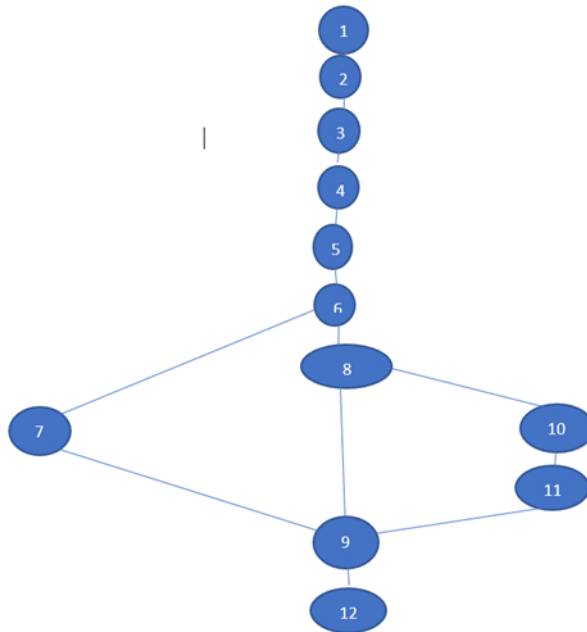
1. CÓDIGO FUENTE

```
int buscarPorID(Registro lista[], int n, char id[]) {  
    for (int i = 0; i < n; i++) {  
        if (strlen(lista[i].id) > 0 && strcmp(lista[i].id, id) == 0) {  
            return i; // Registro encontrado en la posición 'i'  
        }  
    }  
    return -1; // Registro no encontrado  
}
```

2. DIAGRAMA DE FLUJO (DF) PSEINT



3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

R1: 1,2,3,4,5,6,7,9,12

R2: 1,2,3,4,5,6,8,10,11,9,12

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = \text{número de nodos predichados} + 1$
 $V(G) = 1 + 1 = 2$
- $V(G) = A - N + 2$
 $V(G) = 12 - 12 + 2 = 2$

DONDE:

P: Número de nodos predichado

A: Número de aristas

N: Número de nodos

Prueba caja blanca de Requisito N° 3: El programa debera guardar la información en un archivo de texto, en un archivo designado para mayor comodidad.

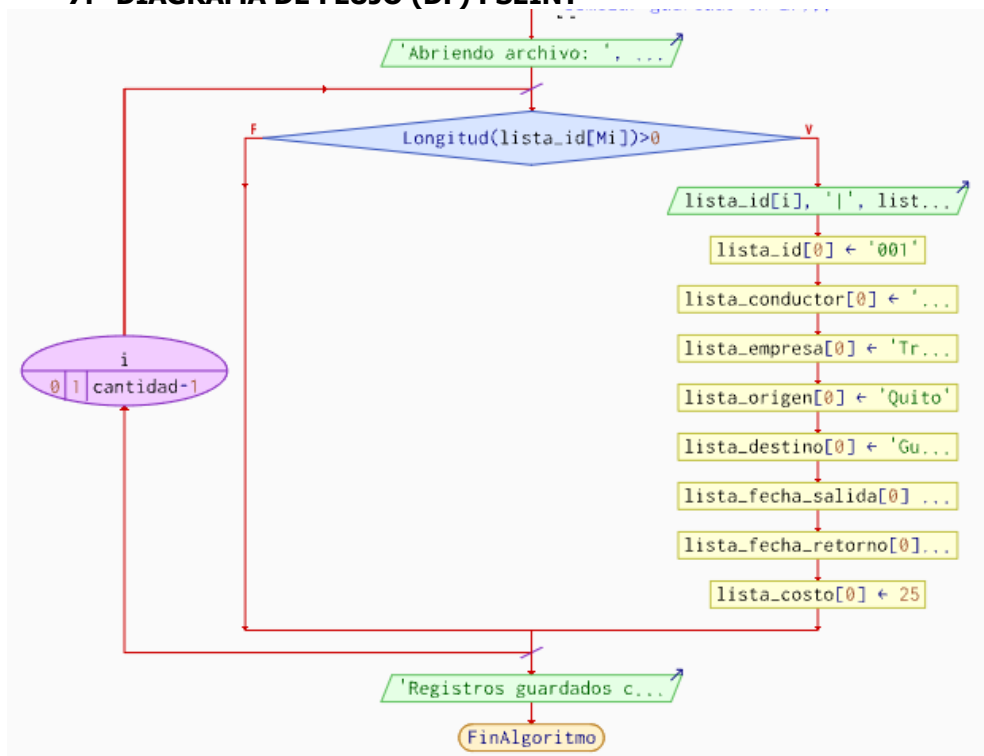
6. CÓDIGO FUENTE

```
// Función para guardar los registros en un archivo de texto
void guardarRegistros(Registro lista[], int cantidad, const char *nombre_archivo) {
    FILE *archivo = fopen(nombre_archivo, "w"); // Abre el archivo en modo escritura ("w")
    if (archivo == NULL) {
        printf("Error: No se pudo abrir el archivo %s para escritura.\n", nombre_archivo);
        return;
    }

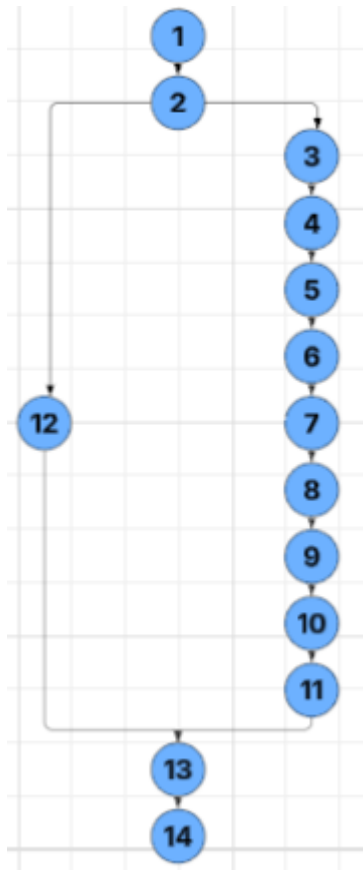
    for (int i = 0; i < cantidad; i++) {
        // Solo guarda los registros que no han sido "eliminados" lógicamente
        if (strlen(lista[i].id) > 0) {
            fprintf(archivo, "%s|%s|%s|%s|%s|%s|%s|%s|\n",
                    lista[i].id,
                    lista[i].conductor,
                    lista[i].empresa,
                    lista[i].origen,
                    lista[i].destino,
                    lista[i].fecha_salida,
                    lista[i].fecha_retorno,
                    lista[i].costo);
        }
    }

    fclose(archivo); // Cierra el archivo
    printf("\nRegistros guardados con exito en '%s'.\n", nombre_archivo);
}
```

7. DIAGRAMA DE FLUJO (DF) PSEINT



8. GRAFO DE FLUJO (GF)



9. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

R1: 1→2→3→4→5→6→7→8→9→10→11→13→14

R2: 1→2→12→13→14

10. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = \text{número de nodos predichados(decisiones)} + 1$ $V(G) = 1 + 1 = 2$
- $V(G) = A - N + 2$
 $V(G) = 14 - 14 + 2 = 2$

DONDE:

P: Número de nodos predichado

A: Número de aristas

N: Número de nodos

Prueba caja blanca de Requisito N° 4: El programa debera cargar el archivo creado con toda la información que contenga, para las demas funciones.

1. CÓDIGO FUENTE

```
// Función para cargar los registros desde un archivo de texto
int cargarRegistros(Registro lista[], int max_cantidad, const char *nombre_archivo) {
    FILE *archivo = fopen(nombre_archivo, "r"); // Abre el archivo en modo lectura ("r")
    if (archivo == NULL) {
        printf("Advertencia: No se encontro el archivo %s. Se iniciara con registros vacios.\n", nombre_archivo);
        return 0; // No se cargó ningún registro
    }

```

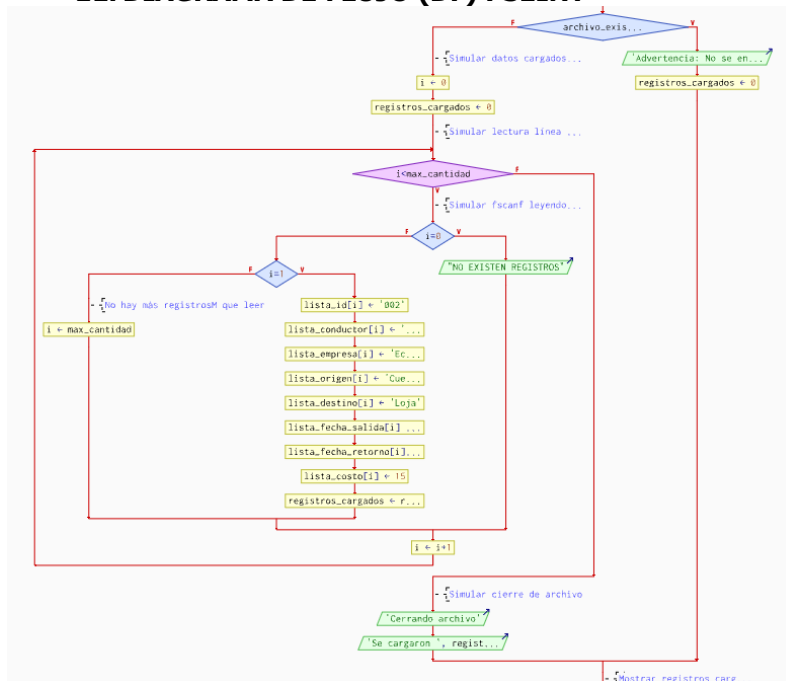
```
    int i = 0;
    // Lee línea por línea, usando '|' como delimitador
    while (i < max_cantidad &&
        fscanf(archivo, "%[^|]|%[^|]|%[^|]|%[^|]|%[^|]|%[^|]|%[^|]|%d\n",
            lista[i].id,
            lista[i].conductor,
            lista[i].empresa,
            lista[i].origen,
            lista[i].destino,
            lista[i].fecha_salida,
            lista[i].fecha_retorno,
            &lista[i].costo) == 8) { // Verifica que se hayan leído
        los 8 campos
        i++;
    }

```

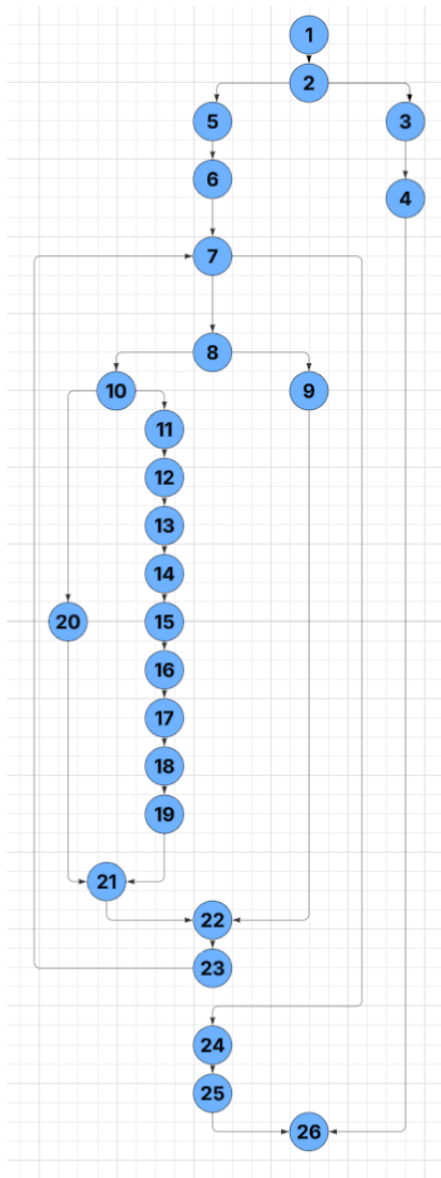
```
    fclose(archivo); // Cierra el archivo
    printf("\nSe cargaron %d registros desde '%s'.\n", i, nombre_archivo);
    return i; // Retorna la cantidad de registros cargados
}

```

11. DIAGRAMA DE FLUJO (DF) PSEINT



12. GRAFO DE FLUJO (GF)



13. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

R1:

R2:

14. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = \text{número de nodos prediados(decisiones)} + 1$ $V(G) = 4 + 1 = 5$
- $V(G) = A - N + 2$ $V(G) = 29 - 26 + 2 = 5$

DONDE:

P: Número de nodos prediados

A: Número de aristas

N: Número de nodos

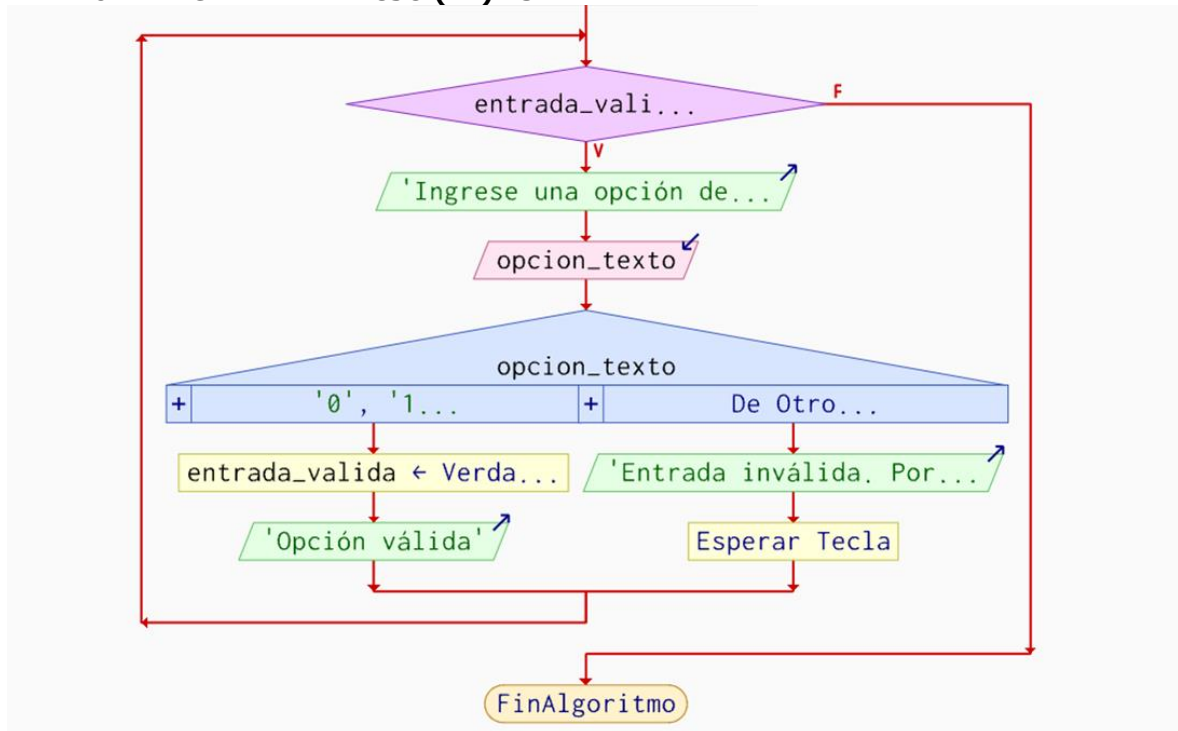
Prueba caja blanca de Requisito N° 5: El programa debera validar la entrada de una

opcion correcta del menú para la ejecución del resto del programa.

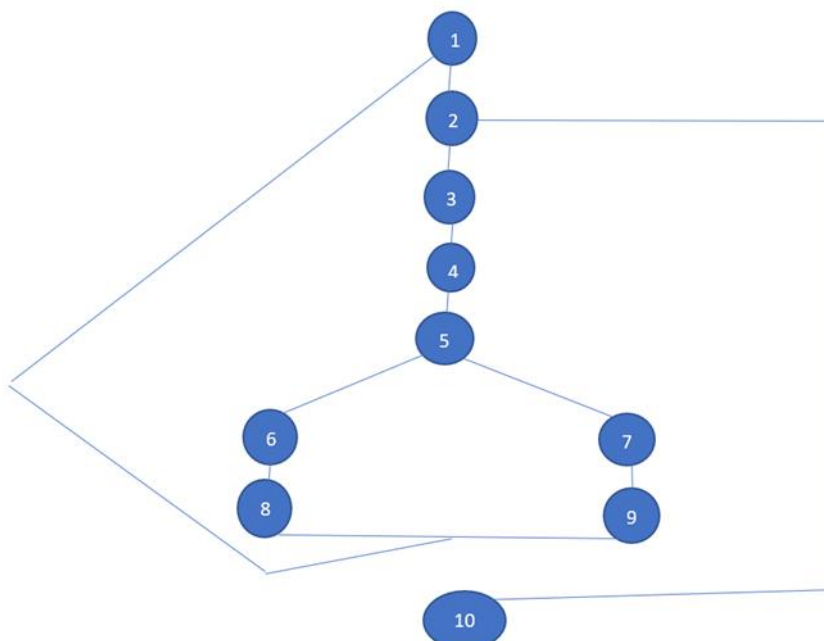
15. CÓDIGO FUENTE

```
if (scanf("%d", &opcion) != 1) { // Validación de entrada numérica para el menú
    printf("Entrada invalida. Por favor, ingrese un numero del menu.\n");
    limpiarBuffer(); // Limpia el bufer si la entrada no es un entero
    pausar();
    continue; // Vuelve al inicio del bucle para mostrar el menu de nuevo
}
```

16. DIAGRAMA DE FLUJO (DF) PSEINT



17. GRAFO DE FLUJO (GF)



18. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

R1: 1,2,10

R2: 2,3,4,5,6,8,1,2,10
R3: 2,3,4,5,6,7,9,1,2,10

19. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = \text{número de nodos} + \text{predicados(decisiones)} + 1$
 $V(G) = 1 + 1 = 2$
- $V(G) = A - N + 2$
 $V(G) = 10 - 10 + 2 = 2$

DONDE:

P: Número de nodos predicado

A: Número de aristas

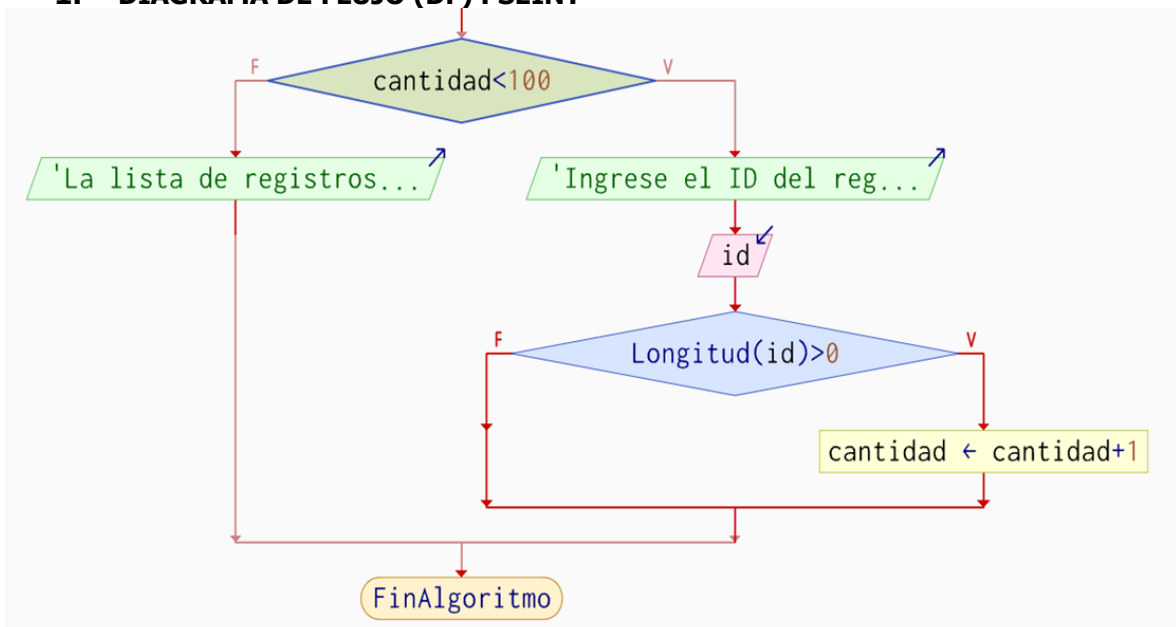
N: Número de nodos

Prueba caja blanca de Requisito N° 6: El programa deberá validar la opción ingresada y verificar que haya espacio para un nuevo registro

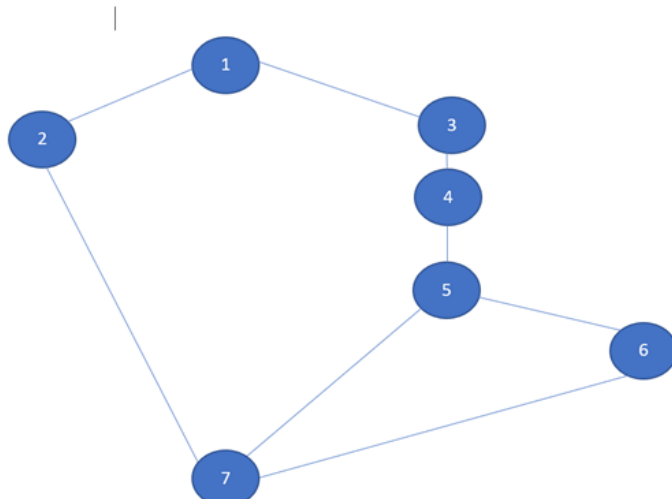
20. CÓDIGO FUENTE

```
switch (opcion) {  
  case 1: // ANADIR REGISTRO  
    if (cantidad < 100) { // Verifica si hay espacio en el arreglo  
      // Pasamos lista y cantidad para la validación de ID único  
      crearRegistro(&lista[cantidad], lista, cantidad);  
      // Solo incrementa la cantidad si el ID no está vacío (registro válido creado)  
      if (strlen(lista[cantidad].id) > 0) {  
        cantidad++;  
      }  
    }  
    else {  
      printf("La lista de registros esta llena. No se pueden anadir mas.\n");  
    }  
    break;  
}
```

1. DIAGRAMA DE FLUJO (DF) PSEINT



21. GRAFO DE FLUJO (GF)



22. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

R1: 1,2,7

R2: 1,2,3,4,5,7

R3: 1,2,3,4,5,6,7

23. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = \text{número de nodos prediados(decisiones)} + 1$
 $V(G) = 2 + 1 = 3$
- $V(G) = A - N + 2$
 $V(G) = 8 - 7 + 2 = 3$

DONDE:

P: Número de nodos prediados

A: Número de aristas

N: Número de nodos

Prueba caja blanca de Requisito N° 7: El programa debera validar el ID, y preguntar si esta de acuerdo con eliminar el registro.

24. CÓDIGO FUENTE

```

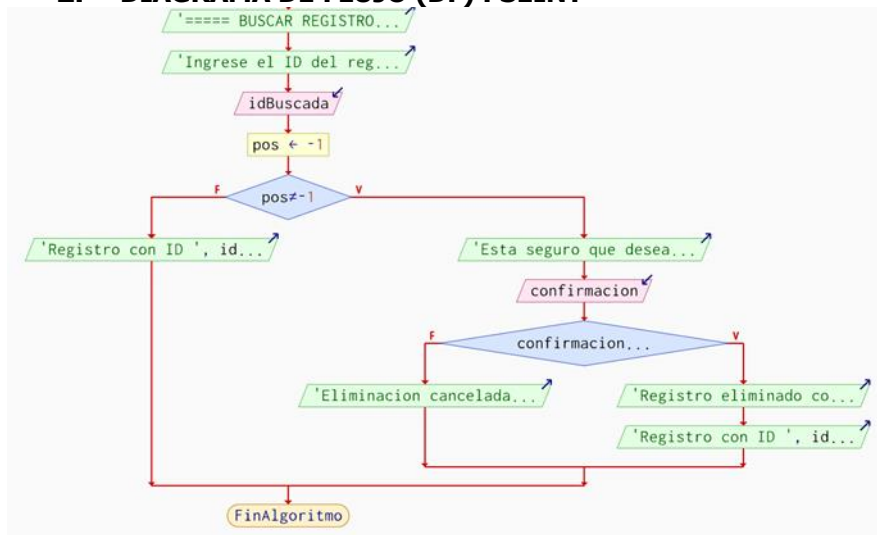
case 3: // ELIMINAR REGISTRO
printf("\n===== BUSCAR REGISTRO PARA ELIMINAR =====\n");
printf("Ingrese el ID del registro que desea eliminar: ");
fgets(idBuscada, sizeof(idBuscada), stdin);
idBuscada[strcspn(idBuscada, "\n")] = '\0';

pos = buscarPorID(lista, cantidad, idBuscada);
if (pos != -1) {
    char confirmacion;
    printf("¿Esta seguro que desea eliminar el registro con ID '%s'? (s/n): ", idBuscada);
    scanf(" %c", &confirmacion); // Espacio antes de %c para consumir el \n pendiente
    limpiarBuffer(); // Limpia el buffer después de scanf

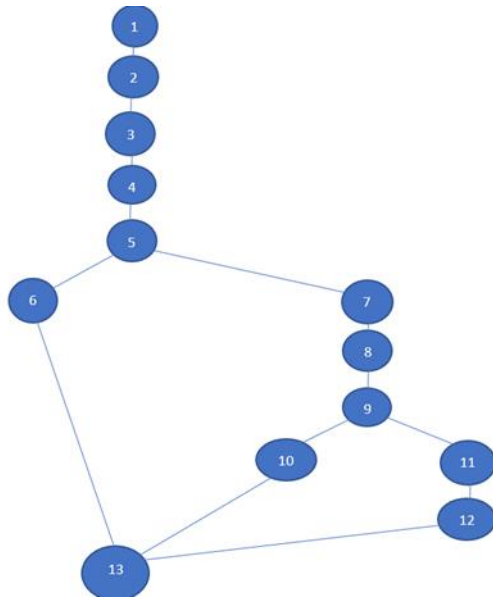
    if (confirmacion == 's' || confirmacion == 'S') {
        eliminarRegistro(&lista[pos]); // "Vaciamos" el registro
        printf("Registro con ID '%s' eliminado con exito.\n", idBuscada);
    } else {
        printf("Eliminacion cancelada. El registro no fue modificado.\n");
    }
} else {
    printf("Registro con ID '%s' no encontrado. No se pudo eliminar.\n", idBuscada);
}
break;

```

2. DIAGRAMA DE FLUJO (DF) PSEINT



25. GRAFO DE FLUJO (GF)



26. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

R1: 1,1,3,4,5,6,13

R2: 1,2,3,4,5,7,8,9,10,13

R3: 1,2,3,4,5,7,8,9,11,12,13

27. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = \text{número de nodos} - \text{predicados(decisiones)} + 1$ $V(G) = 2 + 1 = 3$
- $V(G) = A - N + 2$
- $V(G) = 14 - 13 + 2 = 3$

DONDE:

P: Número de nodos predicado

A: Número de aristas

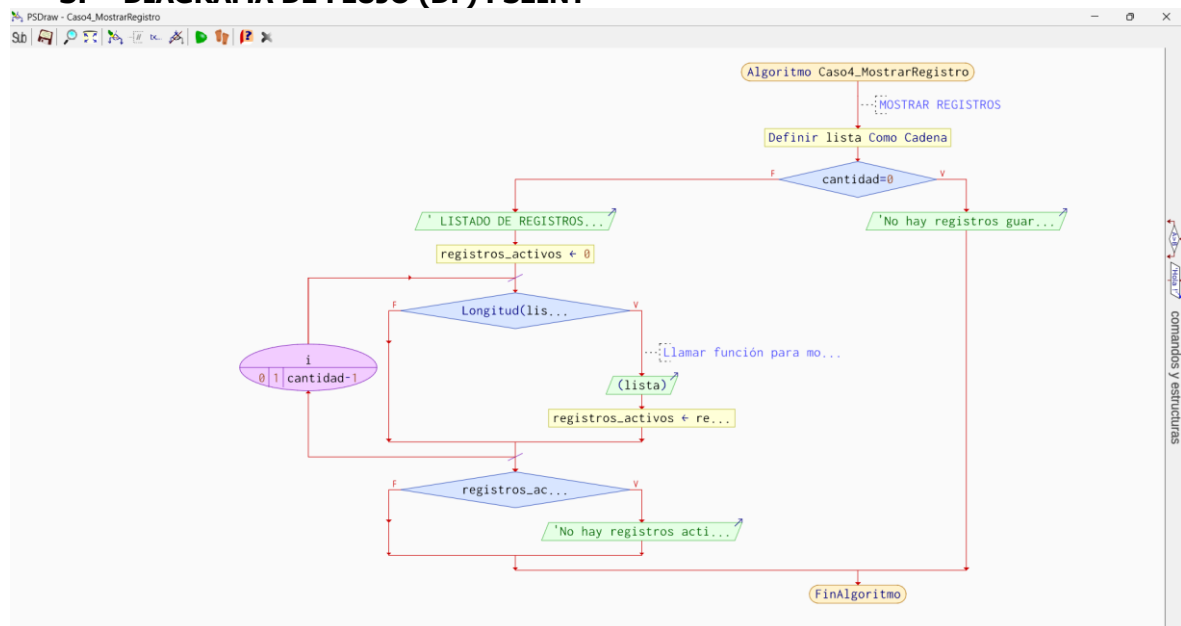
N: Número de nodos

Prueba caja blanca de Requisito N° 8: El programa deberá validar que exista o no los registros para desplegar la información o un mensaje que indique que no existe ningún registro

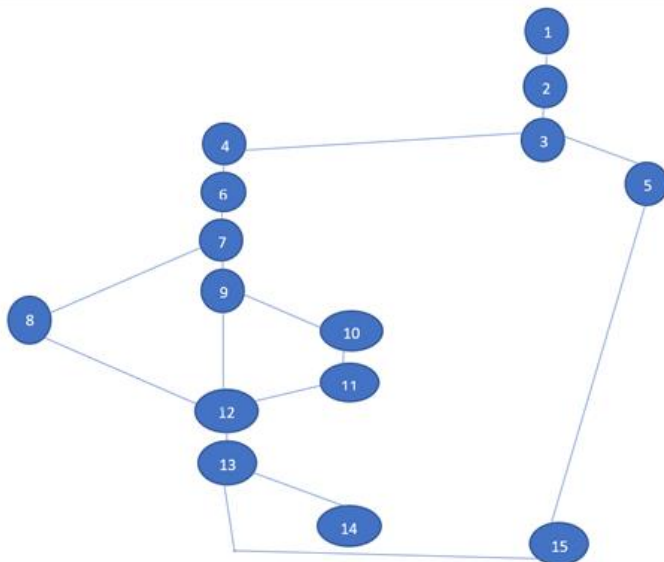
28. CÓDIGO FUENTE

```
case 4: // MOSTRAR REGISTROS
    if (cantidad == 0) {
        printf("\nNo hay registros guardados para mostrar. Anada uno primero.\n");
    } else {
        printf("\n===== LISTADO DE REGISTROS ACTIVOS =====\n");
        int registros_activos = 0;
        for (int i = 0; i < cantidad; i++) {
            if (strlen(lista[i].id) > 0) { // Solo muestra si el ID no esta vacio (registro activo)
                mostrarRegistro(lista[i]);
                registros_activos++;
            }
        }
        if (registros_activos == 0) {
            printf("No hay registros activos para mostrar. Todos han sido eliminados o la lista esta vacia.\n");
        }
    }
    break;
```

3. DIAGRAMA DE FLUJO (DF) PSEINT



29. GRAFO DE FLUJO (GF)



30. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

R1: 1,2,3,5,15

R2: 1,2,3,4,6,7,9,12,13,14,15

R3: 1,2,3,4,6,7,9,10,11,12,13,15

R2: 1,2,3,4,6,7,8,12,13,15

31. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = \text{número de nodos predados(decisiones)} + 1$ $V(G) = 3 + 1 = 4$
- $V(G) = A - N + 2$
 $V(G) = 17 - 15 + 2 = 4$

DONDE:

P: Número de nodos predado

A: Número de aristas

N: Número de nodos